

Sparsity-preserving gradient utility tools for PyTorch

Theodore Barfoot¹, Ben Glocker², and Tom Vercauteren¹

¹ King's College London, London, UK ² Imperial College London, London, UK

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

Summary

The `torchsparsegradutils` package provides gradient-preserving sparse tensor operations for PyTorch (Paszke et al., 2019), addressing the critical limitation that PyTorch's native sparse operations do not support sparse gradients during backpropagation. This package enables memory-efficient optimisation of high-dimensional models by maintaining sparsity patterns throughout the entire forward and backward pass computation, supporting both coordinate list (COO) and compressed sparse row (CSR) formats.

Key features include: (1) memory-efficient sparse matrix multiplication with sparse gradient preservation, (2) sparse triangular and generic linear system solvers, enabling sparse gradients during backpropagation, and multiple algorithmic backends (BICGSTAB, CG, LSMR, MINRES), (3) cross-platform sparse solver wrappers for CuPy (Okuta et al., 2017) and JAX (Bradbury et al., 2018), (4) sparse multivariate normal distributions with LL^T and LDL^T sparse covariance and precision matrix parameterisations with reparameterised sampling methods, and (5) specialised encoders for spatial neighbourhood relationships in volumetric data.

The package addresses PyTorch limitation of dense gradients resulting in memory errors, such as issue #41128, by implementing custom autograd functions that preserve sparsity in gradients, enabling practical training of models with sparse covariance and precision structures on high-dimensional data where dense alternatives become computationally intractable.

Statement of need

Sparse tensors are essential for computationally tractable machine learning on high-dimensional data, yet PyTorch's sparse tensor operations suffer from a critical limitation: gradients are computed in dense format even when the forward pass maintains sparsity. This results in prohibitive memory usage that scales quadratically with problem dimension rather than linearly with the number of non-zero elements (nnz).

For applications requiring sparse covariance modelling—such as medical imaging with millions of voxels, spatial statistics, and large-scale Gaussian processes—dense gradient computation renders optimisation infeasible. A sparse covariance matrix with 1 million dimensions and 0.1% sparsity contains 1 billion non-zero elements, but dense gradients would require storing 1 trillion parameters.

This package solves this fundamental limitation by implementing custom autograd functions that preserve sparsity patterns throughout both forward and backward passes. Our sparse multivariate normal distribution enables optimisation of million-dimensional Gaussian models with memory usage scaling as $O(\text{nnz})$ rather than $O(n^2)$, where n is the dimension of the multivariate distribution.

Beyond memory efficiency, the package addresses the algorithmic challenge of sparse linear system solving by providing multiple iterative solver backends with automatic differentiation

40 support. This enables end-to-end optimisation of complex probabilistic models that would be
41 computationally intractable with existing PyTorch sparse operations.

42 Mathematics

43 Sparse Matrix Operations

44 Sparse Matrix Multiplication

45 The package implements sparse-dense matrix multiplication $\mathbf{C} = \mathbf{AB}$ where $\mathbf{A} \in \mathbb{R}^{m \times n}$ is
46 sparse and $\mathbf{B} \in \mathbb{R}^{n \times p}$ is dense. The forward pass uses PyTorch's native `torch.sparse.mm`,
47 while the backward pass is reimplemented to preserve sparsity patterns in the gradients.

48 Given upstream gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{C}} \in \mathbb{R}^{m \times p}$ from some scalar objective function $\mathcal{L}$, the chain rule
49 gives:

50 **Gradient wrt $\mathbf{B}$ (dense):**

$$\frac{\partial \mathcal{L}}{\partial \mathbf{B}} = \mathbf{A}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{C}}.$$

51 **Gradient wrt $\mathbf{A}$ (sparse):** For a nonzero entry $\mathbf{A}_{ij}$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}_{ij}} = \sum_{k=1}^p \left(\frac{\partial \mathcal{L}}{\partial \mathbf{C}_{ik}} \right) \mathbf{B}_{jk}.$$

52 Sparse Linear System Solvers

53 The package provides multiple approaches for solving sparse linear systems

$$\mathbf{Ax} = \mathbf{B},$$

54 where $\mathbf{A} \in \mathbb{R}^{n \times n}$ is sparse, $\mathbf{B} \in \mathbb{R}^{n \times p}$ is dense (with $p = 1$ for a single right-hand side), and
55 $\mathbf{x} \in \mathbb{R}^{n \times p}$ is the dense solution. We support both direct triangular solves and iterative solvers
56 (CG, BiCGSTAB, LSMR, MINRES). All are differentiable via the implicit function theorem.

57 Given upstream gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} \in \mathbb{R}^{n \times p}$:

58 **Gradient wrt $\mathbf{b}$ (dense):**

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \mathbf{A}^{-\top} \frac{\partial \mathcal{L}}{\partial \mathbf{x}}.$$

59 **Gradient wrt $\mathbf{A}$ (sparse):** The dense form would be

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}} = - \left(\mathbf{A}^{-\top} \frac{\partial \mathcal{L}}{\partial \mathbf{x}} \right) \mathbf{x}^\top.$$

60 For a nonzero entry $\mathbf{A}_{ij}$ this becomes

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}_{ij}} = - \sum_{k=1}^p \left(\mathbf{A}^{-\top} \frac{\partial \mathcal{L}}{\partial \mathbf{x}} \right)_{ik} \mathbf{x}_{jk}$$

61 Sparse Multivariate Normal Distributions

62 The package implements sparse multivariate normal distributions $\boldsymbol{\eta} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \equiv \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Omega}^{-1})$
63 with two parameterisations for efficient sampling. Both methods transform standard normal
64 samples $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ into samples from the desired multivariate normal distribution:

65 LL^T Parameterisation: Sparse lower triangular matrices L with positive diagonals:

$$\eta_{\Sigma} = \mu + L_{\Sigma}\epsilon, \quad \eta_{\Omega} = \mu + L_{\Omega}^{-T}\epsilon$$

66 LDL^T Parameterisation: Sparse unit lower triangular matrices L and diagonal D :

$$\eta_{\Sigma} = \mu + LD^{1/2}\epsilon, \quad \eta_{\Omega} = \mu + L_{\Omega}^{-T}D_{\Omega}^{-1/2}\epsilon$$

67 We store L without its diagonal (strictly lower), and treat it as unit lower-triangular at use
68 time. The LDL^T parameterisation provides superior numerical stability for precision matrices
69 by avoiding strict positive definiteness constraints.

70 Acknowledgements

71 The authors acknowledge the PyTorch development team for providing the foundational
72 sparse tensor infrastructure. We thank the SciPy (Virtanen et al., 2020), CuPy (Okuta
73 et al., 2017), and JAX (Bradbury et al., 2018) communities for high-performance sparse
74 linear algebra implementations. Algorithm implementations adapt and extend methods from
75 pykrylov (BICGSTAB), cornellius-gp/linear_operator (CG, MINRES), and pytorch-minimize
76 (LSMR) (Saad, 2003). We thank Floris Laporte for his excellent tutorial on implementing
77 sparse linear system solvers in PyTorch (Laporte, 2020), which provided valuable insights for
78 gradient computation strategies. This work was supported by the Wellcome/EPSRC Centre
79 for Interventional and Surgical Sciences and the School of Biomedical Engineering & Imaging
80 Sciences, King's College London.

81 References

- 82 Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G.,
83 Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable
84 transformations of Python+NumPy programs*. <https://github.com/jax-ml/jax>
- 85 Laporte, F. (2020). *Solving sparse linear systems in PyTorch*. [https://blog.flaport.net/
86 solving-sparse-linear-systems-in-pytorch.html](https://blog.flaport.net/solving-sparse-linear-systems-in-pytorch.html).
- 87 Okuta, R., Unno, Y., Nishino, D., Hido, S., & Loomis, C. (2017). CuPy: A NumPy-compatible
88 library for NVIDIA GPU calculations. *Proceedings of the Workshop on Machine Learning
89 Systems (LearningSys) at NeurIPS*. [https://learningsys.org/nips17/assets/papers/paper_
90 16.pdf](https://learningsys.org/nips17/assets/papers/paper_16.pdf)
- 91 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin,
92 Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-
93 performance deep learning library. *Advances in Neural Information Processing Systems*, 32,
94 8024–8035.
- 95 Saad, Y. (2003). *Iterative methods for sparse linear systems* (2nd ed.). SIAM. [https://
96 www-users.cse.umn.edu/~saad/IterMethBook_2ndEd.pdf](https://www-users.cse.umn.edu/~saad/IterMethBook_2ndEd.pdf)
- 97 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
98 Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0:
99 Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3),
100 261–272. <https://doi.org/10.1038/s41592-019-0686-2>